

- PL is an artificial language used to communicate with computers(Machines).

- PL is used to build

software.



Source Code or
Program

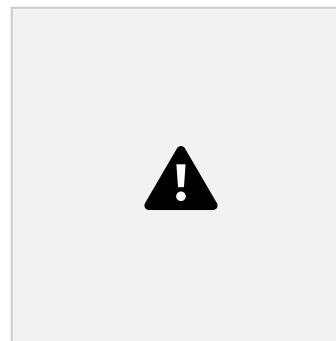
Translator
(Compiler Or

Interpreter)

Machine Code or

Software

Computer



Created By:

- ❖ Initiated in June 1991 and First version came in 1995.
- ❖ Developed by Sun Microsystems.
- ❖ Best suited for Security and Platform Independency.
- ❖ First name: Oak
- ❖ Second name: Green
- ❖ Final name: Java (came from Java Coffee of Indonesia).
- ❖ Syntax came from C and C++
- ❖ Oracle Corporation acquired Sun Microsystems in 2010.









MORE
THAN

95% OF THE ENTERPRISES ARE USING JAVA.



MORE THAN 3 BILLION DEVICES ARE USING JAVA.



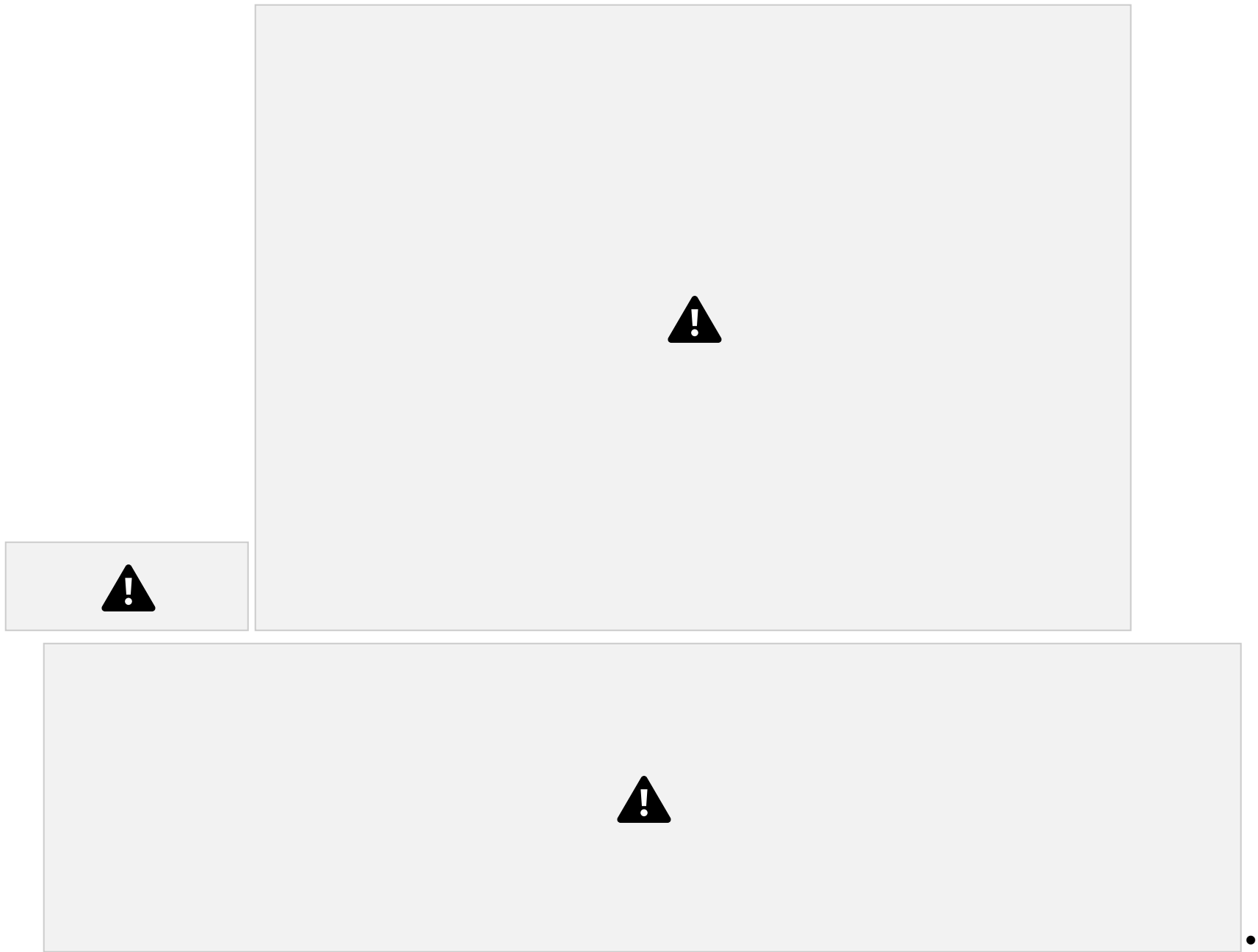
JAVA IS **#1 CHOICE** FOR BACKEND DEVELOPMENT.











JRE is an environment where the java's byte code execute.

- **JVM** is the machine that executes the java's byte code.
 - **JIT(Just-In Time)** is an interpreter used by JVM to interpret java's byte code to native code(machine code) that gives output.
 - **JAVAC** is the compiler that compiles the source code and generates byte code.
- JDK** is the kit which contains the JRE, JVM and JAVAC.



Comment is a statement that is ignored by the compiler at the time of compilation. Types of comment:

- **Single line Comment:**



// this is
single line
comment

- **Multi**

line Comment:

```
/*
```

```
This is multiple line comment
```

```
Line 2
```

```
Line 3
```

```
*/
```







**Every Individual units in a program
is TOKEN**



A word created by language for particular job that can not be changed by programmer is a keyword.







- Keyword is a reserved word.
- Keyword is always be in small letters
- Keyword does not contain digit or special letter (like @ & # etc).
- Keyword list varies according to versions of java. The **goto** and **const** keywords are not in use. • **true**, **false**, and **null** might seem like keywords, but they are actually literals and reserved words.





Identifier is a word created by a programmer for variable-name , function-name, class-name, interface-name and package-name to use them further.





Rules:

1. It can only be from Alphabets(a-z, A-Z) and Digit(0-9) and

underscore(_) and any currency symbol [like: \$, ₹] . 2. It can not start from digit.

3. Only Underscore is not allowed.

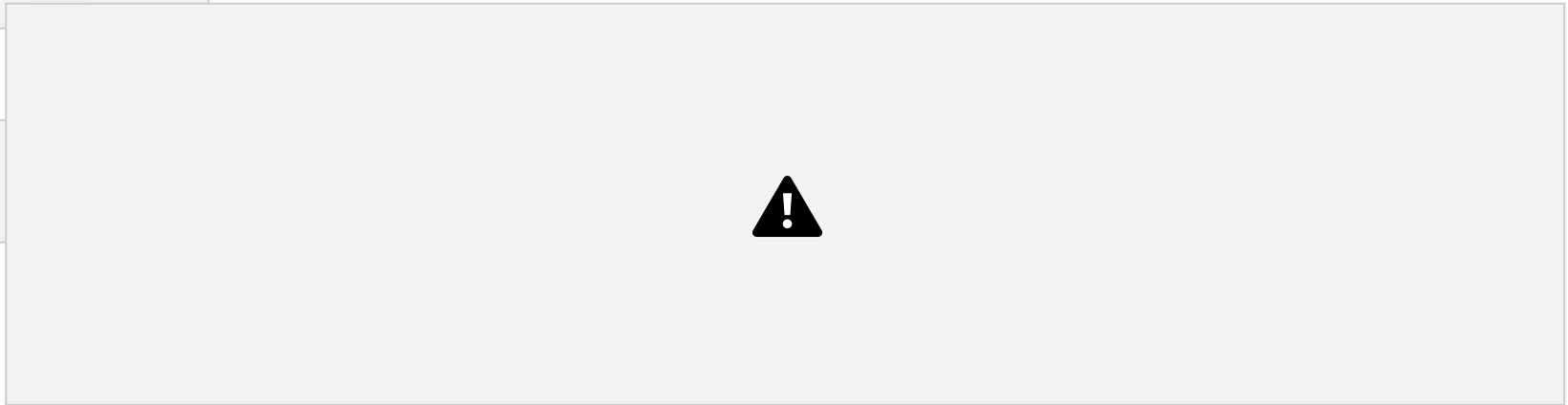
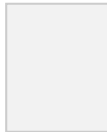
4. Keyword can not be an Identifier.



Operator is a symbol

used to do operation on data.



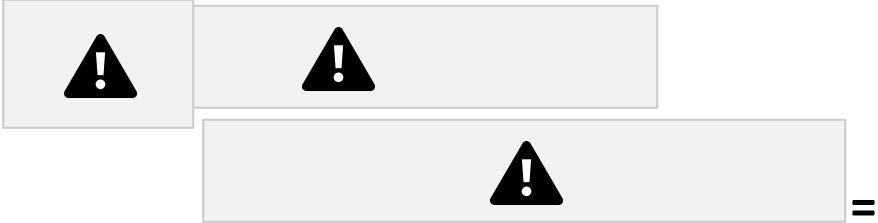


**Shorthand/Compound
Increment-Decrement**

Assignment



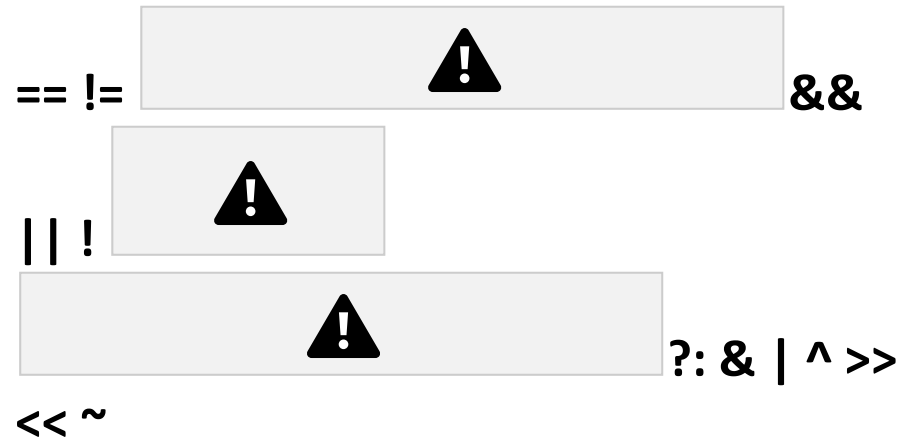
Arithmetic



Conditional Bitwise

%= ++ --

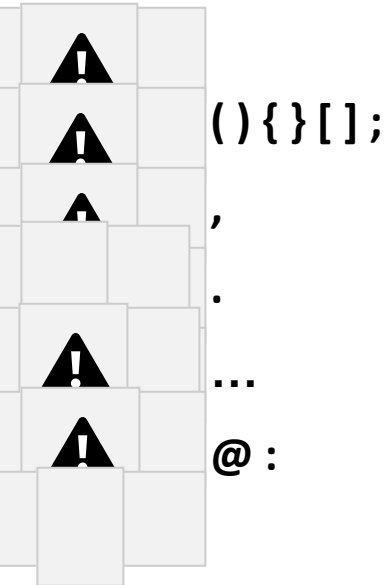


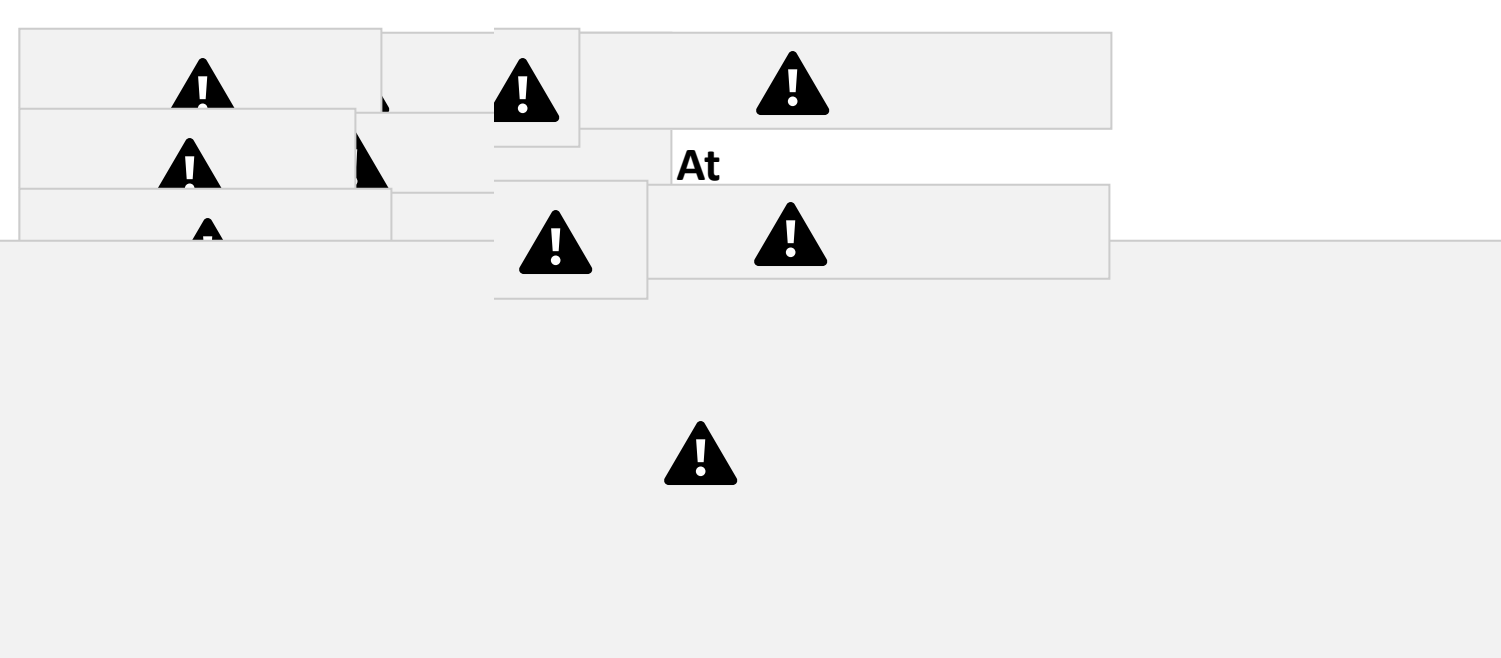


Separator is a symbol

that is used to do separation in code.







Round Bracket Curly Colon

Bracket Square

Bracket Semicolon

Coma



Dot



Triple Dot



Double Colon



Literal is the constant data in a

program.











- In between the
keyword and **keyword**
identifier and **identifier**



keyword
and

identifier

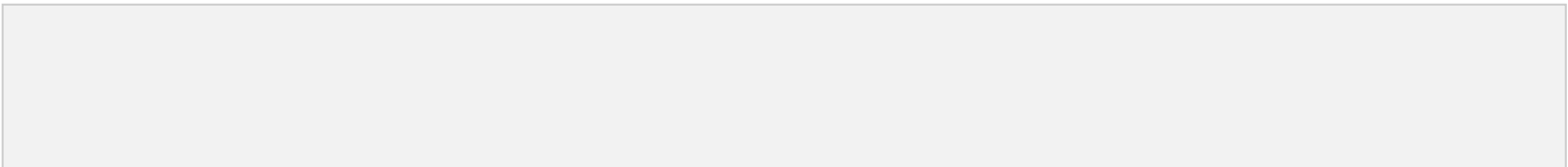
identifier and **keyword**

gap(space or new line) is compulsory. • In

between other tokens gap is optional.



Variable is just a container that





contains data.

101 int roll= 101

;

Rahul String name= "Rahul" ; 5.9 double

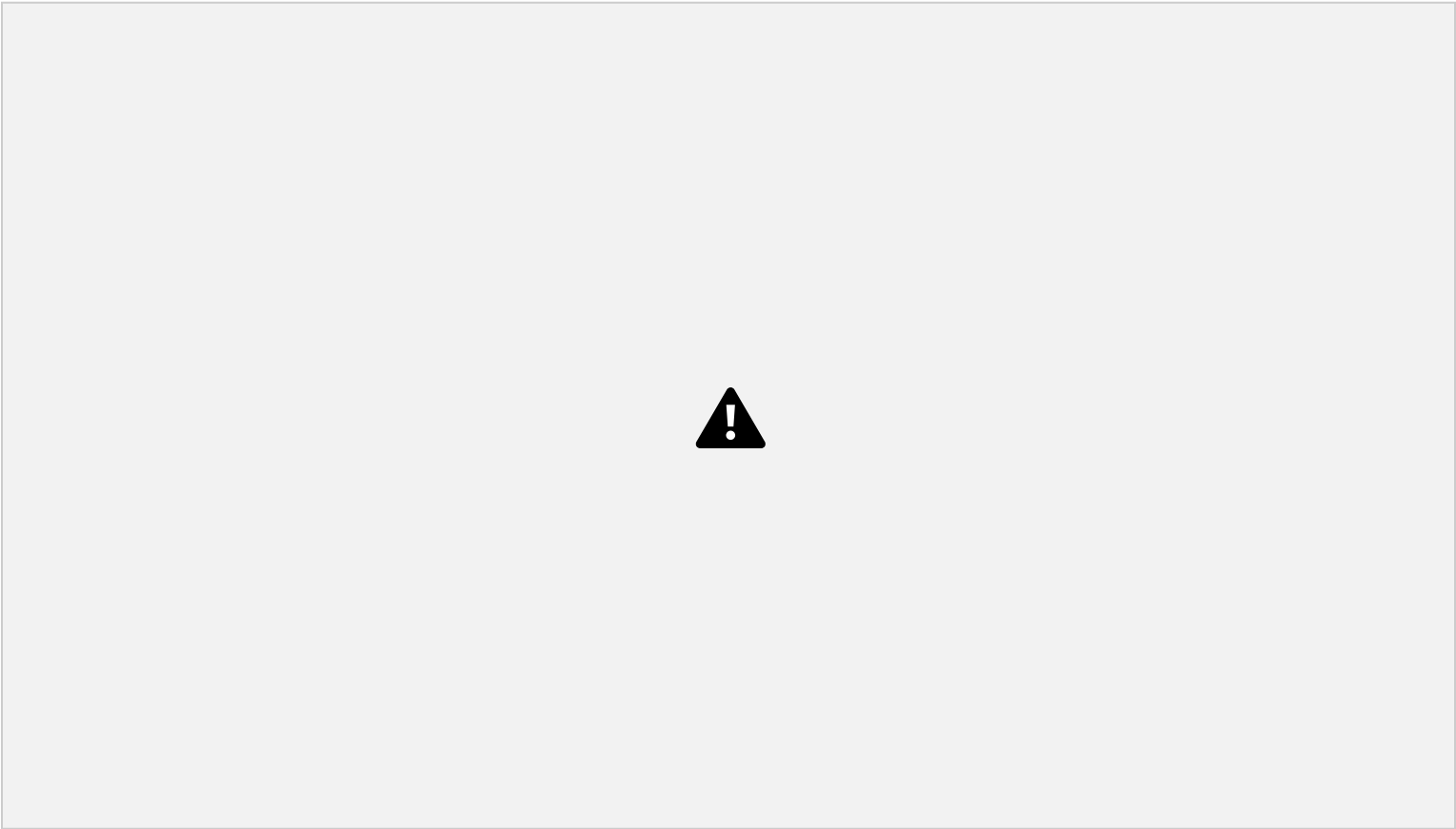
height= 5.9 ;





Data type is just a word used to specify the type of the data, size of the data and range of the data. Java has two types of Data Types: **Primitive** and **Non-Primitive** .











Converting the type of data into another

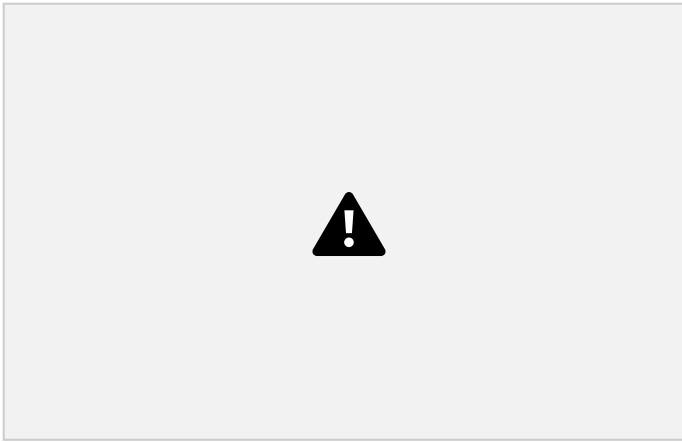
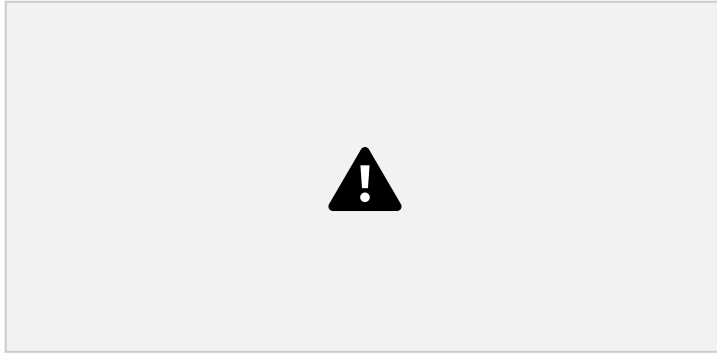
data type.



Implicit Typecasting

Explicit Typecasting





Upcasting

(Widening) Downcasting (Narrowing)







Operator is a symbol used to do operation on data.

Shorthand/Compound



Increment-Decrement



Assignment



Relational



Arithmetic



Logical





Conditional Bitwise



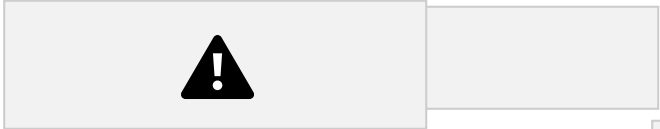
+ - * / %



++ --



< <= > >= == !=



&& || !



=

?: & | ^ >> << ~



=



int a;

a = 10;



int

a = 10;



```
int a = 10, b = 15; 10 = a;
```

```
int a, b;
```

```
a = b = 10;
```





+ - * / %



a = 10 -



int a;

a = 10 +
2; SOP(a):



2;
SOP(a):

int a;

int a;
a = 10 *
2; SOP(a);

a = 10 /
2;
SOP(a):



int a;

a = 10 %
2; SOP(a);



int a;



$+=$ $-=$ $*=$ $/=$ $\%=$

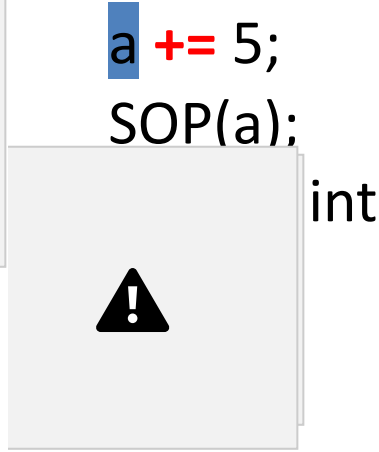




```
int a=10;
a = a / 5;
SOP(a);
```

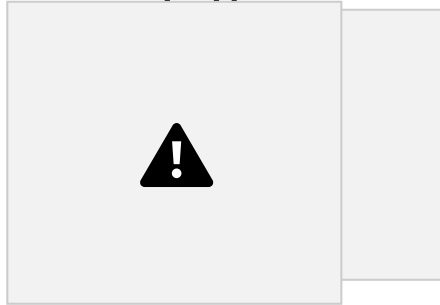


```
int a=10;
a /= 5;
```

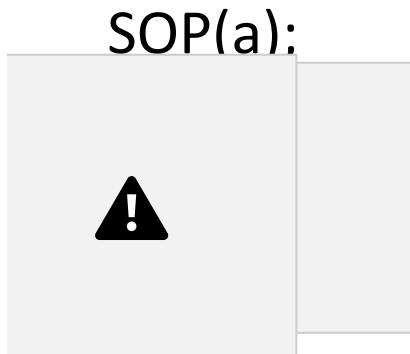


```
a += 5;
SOP(a);
```

int



```
int a=10;
a = a + 5;
SOP(a);
```

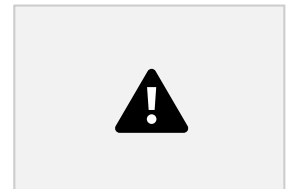


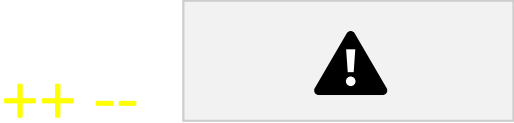
```
SOP(a);
```

```
int a=10;
```

```
a=10; a
/= 5;
SOP(a);
```

```
int a=10;
a /= 5;
SOP(a);
```







```
a=10; a  
= a - 1;  
SOP(a);
```

```
int a=10;  
a = a + 1;  
SOP(a);
```



```
a ++;  
SOP(a);
```

```
int a=10;  
a ++ + ;
```



```
int  
a=10; a  
= a - 1;  
SOP(a);
```

```
int a=10; 10 ++ ;  
a = a + 1;  
SOP(a);
```



```
int a=10;  
++ a;  
SOP(a);
```

```
SOP(a);
```



```
int
```

```
a=10;  
a --;
```



```
int a=10;
```

```
int
```

x

Prefix



int

Postfi

a=10;

-- a;

SOP(a);

SOP(a);



< <= > >= == !=



```
int a=10,b=5;  
boolean  
c=a<b; SOP(c);
```

```
int a=10,b=5;  
boolean  
c=a>b; SOP(c);
```

```
int a=10,b=5;  
boolean  
c=a!=b; SOP(c);
```



```
int a=10,b=5;  
boolean  
c=a<=b; SOP(c);
```



```
int a=10,b=5;  
boolean  
c=a==b; SOP(c);
```

```
int a=10,b=5;  
boolean  
c=a>=b; SOP(c);
```



SOP(c);



int a=10,b=5;
boolean c=a<=b;



&& || !



int a=10,b=5;

true && true

true

boolean c=a>b &&

SOP(c); true &&

false false

int

a=10,b=5;
boolean c=a<b &&
b<3; SOP(c);

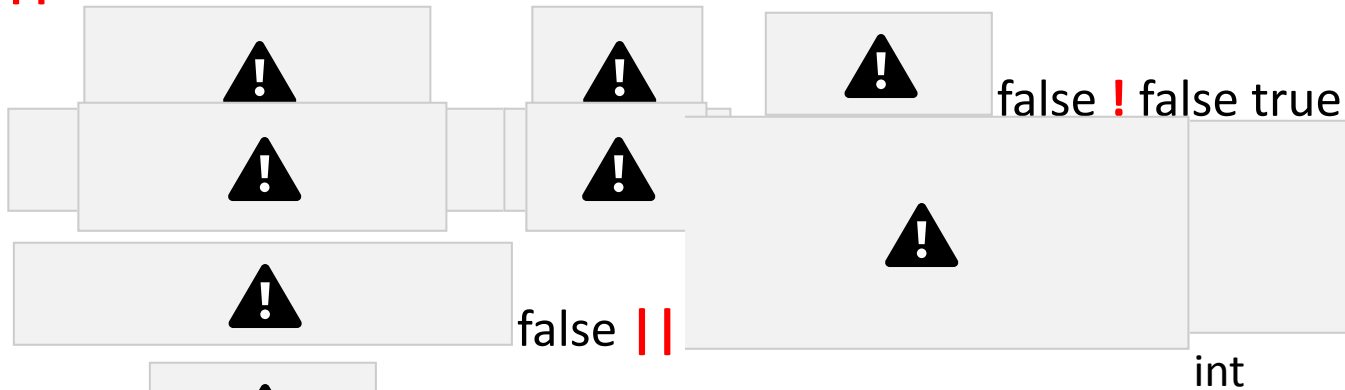
&& true false

false && false false



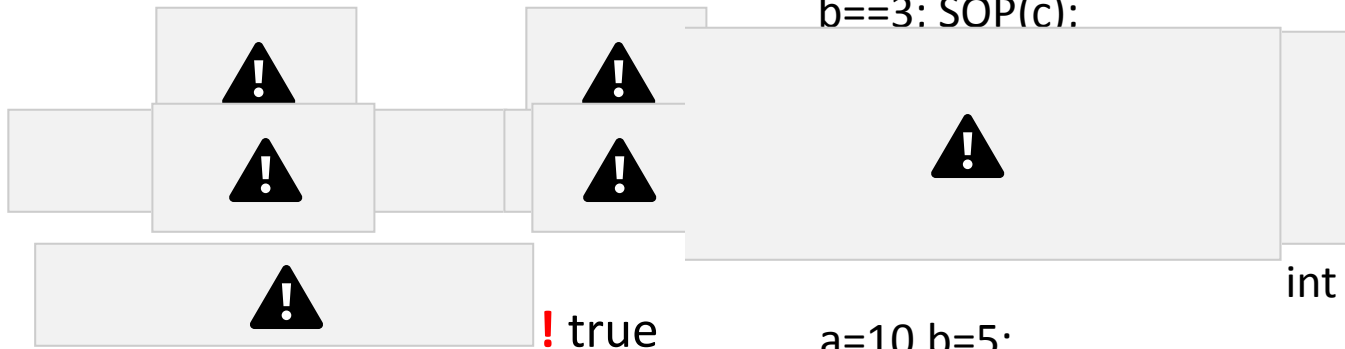
true || true true true
|| false

```
int a=10,b=5;
boolean c=a<b ||
b<3; SOP(c);
```



true true false ||
false false

```
a=10,b=5;
boolean c=a>b ||
b==3; SOP(c);
```



a=10,b=5;

boolean c=! (a>b) ||

!(b==3); SOP(c);



?:



Exp. 2



:



Exp. 3



Exp. 1



?





false



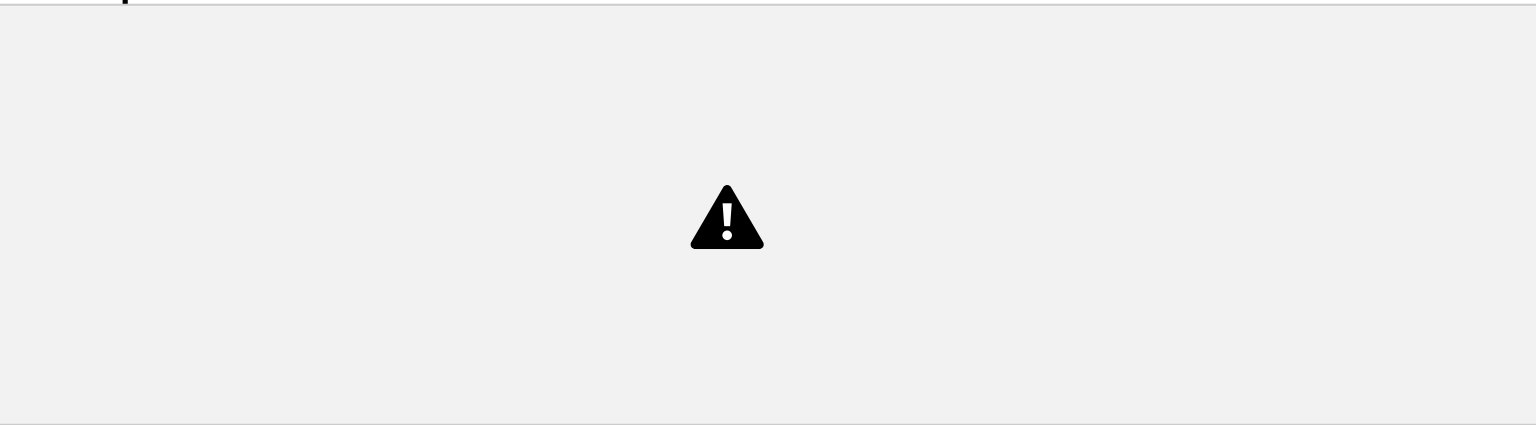
true



Exp. 3



Exp. 2



int a=10,b=5;



int c=a<b ? a+12/3-2 : b%5+a-7;
SOP(c);



& | ^ >> << ~





000....000101



000....000100 int
a=12;

int c=a >>
1; SOP(c);



int a=12,b=5;
int c=a & b;
SOP(c):



000....001100



000....001100
0000....00110



int a=12,b=5;
int c=a | b;
SOP(c);



int c=a <<
1: SOP(c):



000...001100



000...011000



int a=12,b=5;
int c=a ^ b;
SOP(c):



000...001100



000...001100
000...000101



000...001101 int
a=12;

000....000101



a=12;

int c= ~ a;

SOP(c);



$\sim a = -(a+1)$



000....001001 int







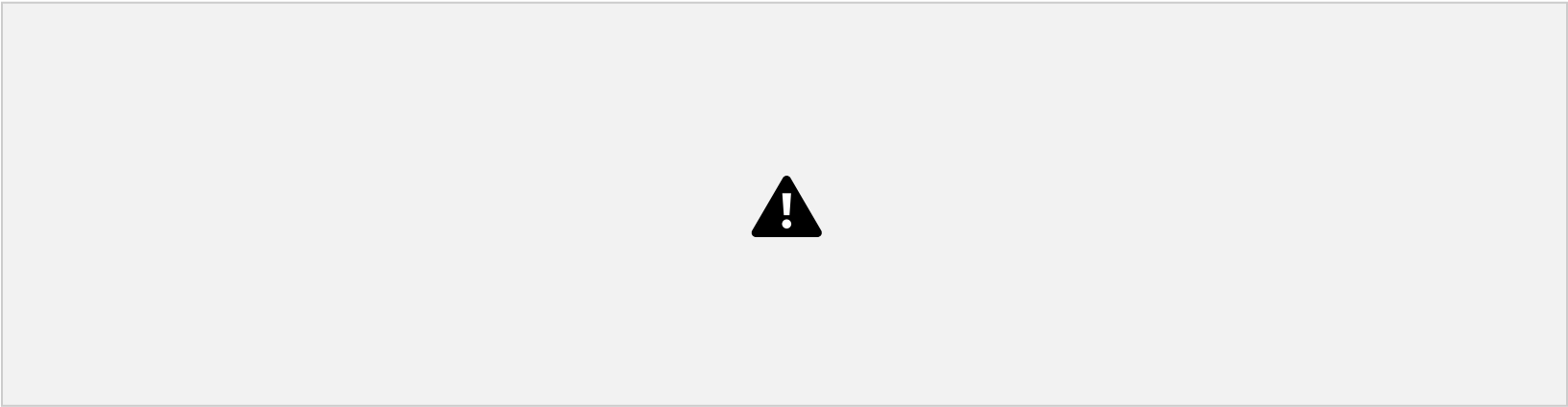


















```
java.util.Scanner sc=new java.util. Scanner( System.in );
```



int age

```
=sc.nextInt();
```

```
double height =sc.nextDouble();
```

```
String name =sc.next();
```



Println function is Expression based.

So we have to understand expression rules.



Integer

Integer



Character Integer *
Integer Character
/

$7+2 = 9$
 $7/2 = 3$ (not 3.5)
 $'a'+10 = 107$
 $10+ 'a' = 107$
 $'h'*10 = 1040$

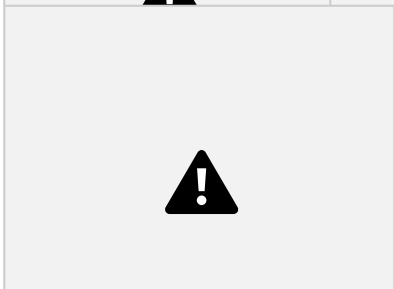


%

Character Character



For Example:





Integer



Floating Point +

Floating Point Integer -



Floating Point



7.0+2 = 9.0

7/2.0 = 3.5

'a'+10.1=107.1

*

Floating Point Character /

10.0+ 'a'=107.0

'h'*10.0=1040.0



Character

Floating Point %



Integer



Floating Point



Floating Point

For Example:



Integer



String



Integer



String

Floating Point
Character



+



String





Floating Point

Character



String

+



String



String





For Example:



5 + "hi" =
"5hi"
"hi"+5 =
"hi5";
4+2+ "hi"
= "6hi"
5.8+ "hi"
= "5.8hi";

5+6+ "hi" + 5+6=

"11hi56" "hello"+ "hi" =
"hellohi"

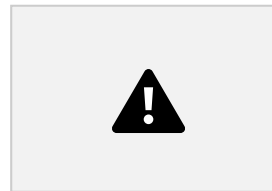
"hello"+ 'a'= "helloa" (not
hello97) 'a' + "hello" = "ahello"



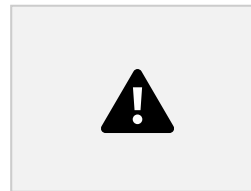
Integer



/
%



Floating Point
Character



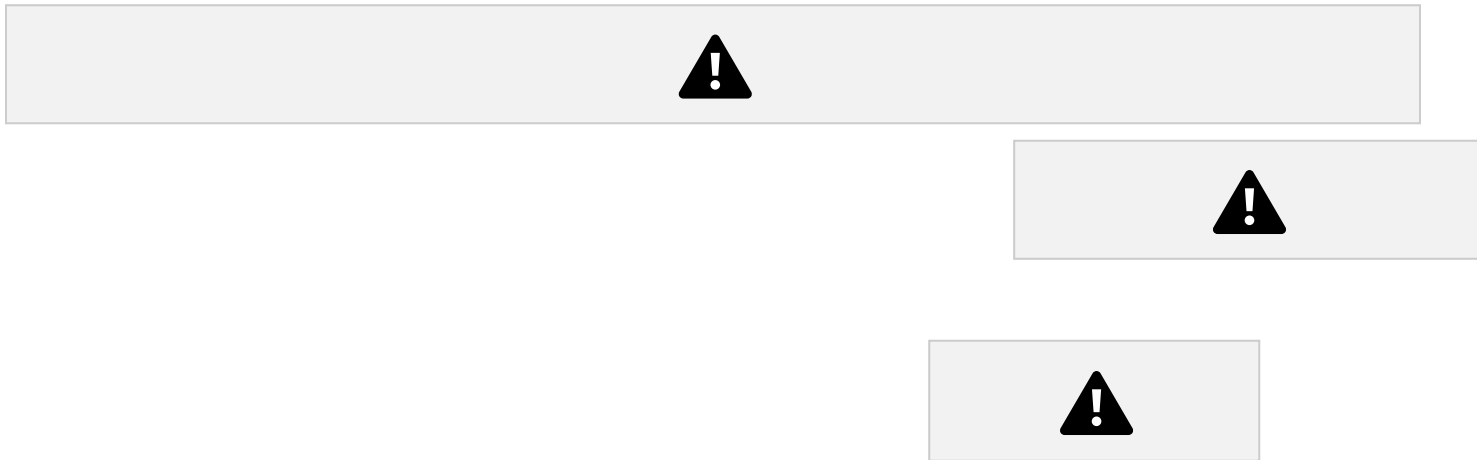
String



String



-
*



- Decision Making is used to execute a block of code on a decision. if the decision is satisfied then code will execute otherwise something else will execute or nothing execute.

• Types
of
decision

making:





if

```
(expression) { // statements  
}
```



Expression -> **true** int age=25;

```
if (age>18) {  
  // statements  
}
```

// statements after if



Here, **expression** must be boolean.

Boolean expression returns either true or false.



Expression -> **false**

```
if (age>18) {  
    // statements  
}  
// statements after if
```

```
int age=15;
```





Expression -> **true** int

age=25;

if (age>18) {

// statements of if

}

else {

// statements of else

}

// statements after if else



Expression -> **false** int

age=15;

if (age>18) {

// statements of if

}

else {

// statements of else

}

// statements after if else



```
int age=15;
```

```
if (age>60) {
```

```
//
```



statements of if

}

else if (**age>18**) {

// statements of else if }

else {

// statements of else

}

// statements after if else if





```
switch (value/expression) {  
case value1 :  
    // statements of case  
break;  
case value2 :  
    // statements of case
```

break;

default :

```
// statements of default }  
// statements after switch
```



- Switch works with



the **byte**, **short**, **char**, and **int**, **String** class, and few wrapper classes **Byte**, **Short**, **Character** and **Integer**.

- Switch does not work with

float, double, boolean and long.





- Loop is used to execute a block of code for multiple times. •
- Loop is also known as Iterative statement or Iteration.
- Write the code once and execute it many times.

• Types
of loop:





Without Loop



SOP("Rahul");

SOP("Rahul"); SOP("Rahul");
SOP("Rahul"); SOP("Rahul");
SOP("Rahul"); SOP("Rahul");
SOP("Rahul"); SOP("Rahul");
SOP("Rahul");

With Loop



```
for ( int a=1 ; a<=10 ; a++ ) {  
    }
```

```
SOP("Rahul");
```

```
}
```

```
for ( Initialization ;  
    TestExpression ; Update ) {  
    // statements
```

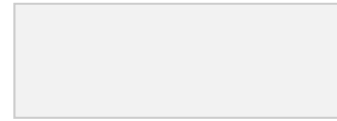




Without

Loop

SOP("Rahul");



With
Loop
t a=1;



SOP("Rahul");
SOP("Rahul");

SOP("Rahul");
SOP("Rahul");



```
while ( SOP("Rahul");  
TestExpression) { // a++;  
statements }
```



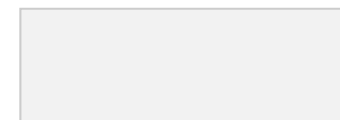
```
while ( a<=5 ) {
```



Without Loop



SOP("Rahul");
SOP("Rahul");



With

do {



SOP("Rahul");

SOP("Rahul"); a++;

SOP("Rahul");

} while (a<=5);

SOP("Rahul");



do {

// statements

} while (

TestExpression) ;



```
for( init; Test-Exp; Update){ // statements  
  if (BreakCondition) {
```



```
// statements
```

```
break;  
}  
// statements  
}
```

```
while (Test-Exp){  
  // statements  
  if (BreakCondition) {  
    // statements  
    break;  
  }  
  // statements
```

```
}
```



```
do{  
  // statements  
  if (BreakCondition) { // statements  
    break;  
  }  
  // statements  
} while (Test-Exp);
```



```
// statements  
continue;  
}  
// statements  
}
```

```
for( init; Test-Exp; Update){  
  // statements  
  if (ContinueCondition) {
```

```
while (Test-Exp){  
  // statements  
  if (ContinueCondition) {  
    // statements  
    continue;  
  }  
  // statements  
}
```



```
do{  
  // statements  
  if (ContinueCondition) { // statements  
    continue;  
  }  
  // statements  
} while  
(Test-Exp);
```





// statements
}



for(init; Test-Exp; Update){ // statements

for(init; Test-Exp; Update){

// statements

}



```
Update){ // statements  
abc: for( init; Test-Exp; Update){
```

```
xyz: for( init; Test-Exp;
```

```
// statements
if (BreakCondition) {
// statements
break xyz;
}
// statements
}
// statements
}
```



xyz: for(init; Test-Exp;

```
Update){ // statements
abc: for( init; Test-Exp; Update){
// statements
if (BreakCondition) { //
statements
break; //or //break abc; }
// statements
}
// statements
}
```





```
}  
// statements  
}
```



```
xyz: for( init; Test-Exp;  
Update){ // statements  
abc: for( init; Test-Exp; Update){
```

```
xyz: for( init; Test-Exp;
```



```
// statements  
if (BreakCondition) {  
// statements  
continue xyz;  
}  
// statements
```

```
Update){ // statements  
abc: for( init; Test-Exp; Update){  
// statements  
if (BreakCondition) { //  
statements  
continue;
```

```
//continue abc;
```

```
}
```

```
// statements
```

```
}
```

```
// statements
```

```
}
```





}

SOP("I Love My INDIA");
SOP("I Live in INDIA");
SOP("I am Happy");
SOP("Bye");
SOP("I Love My INDIA");
SOP("I Live in INDIA");
SOP("I am Happy");



PS

PSVM(-){

SOP("Hello");
SOP("I Love My INDIA");
SOP("I Live in INDIA");
SOP("I am Happy");
SOP("Hi");



```
void display(){
```

```
SOP( "I Love My INDIA" );
```

```
SOP( "I Live in INDIA" );
```

```
SOP( "I  
am  
Happy"  
);  
}
```

```
PSVM(-){
```

```
SOP( "Hello" );
```

```
display();
```

```
SOP( "Hi" );
```

```
display();
```

```
SOP( "Bye" );
```

```
display();
```

```
}
```





Return Type



Function Name



Argument



```
void display ( ) {  
    SOP( "I Love My  
INDIA" );  
    SOP( "I Live in  
INDIA" );  
    SOP( "I am Happy" );  
}
```

}



Function Body



Types of Functions:

- Function with No Arguments and No Return Type •



- Function with Arguments but No Return Type •
- Function with No Arguments but Return Type •
- Function with Arguments and Return Type •
- Function with simple Return keyword



```
public class Demo{  
    public static void  
    add(){  
        int a=10,b=20,r=a+b;  
        SOP( "Sum: "+r );  
    }  
    public static void  
    main(String[] s){  
        add();  
    }  
}
```





```
public static void add(int a, int b){  
    int r= a+b;
```

```
public class Demo{
```

```
    SOP( "Sum: "+r );  
}
```



```
public static void main(String[]  
    s){ add(12,34);  
}  
}
```



```
public class Demo{  
    public static int add(){
```

```
int a=10,b=20,r=a+b;
```

```
return r;  
}
```



```
public static void main(String[]  
s){ int r=add();  
SOP( "Sum: "+r );  
}  
}
```





public class Demo{

public static **int** **add**(**int** a, **int**
b){ **int** r= a+b;

return r;
}

```
public static void main(String[]  
    s){ int r=add(12,34);  
    SOP( "Sum: "+r );  
}  
}
```



```
public class Demo{
```

```
public static void xyz(int a, int  
    b){ SOP( "Hello" );
```



```
if(a==b){  
    return;  
}  
SOP( "Hi" );  
}
```

```
public static void main(String[]  
    s){ xyz(10,20);  
}  
}
```





```
public static void PSVM(-){  
    m(){ int x=10;  
        z=10;  
        SOP(x);  
        SOP(z); // error  
        m();  
    }  
}
```



